Institute of Technology of Cambodia
Faculty of Electrical Engineering
Génie Informatique et Communication

GICI4OS · OPERATING SYSTEMS · LAB 3

Wildcards, Hard Links and Symbolic Links

Lab 3 guide: select files with patterns, then see what a rename does to a hard link and to a symbolic link.



Heng Rathpisey · Department GIC · Academic year 2025–2026

Learning objectives

1. Explain which filenames a Bash wildcard selects, and what quoting changes.
2. Use inode numbers to tell a hard link from a separate copy.
3. Explain and repair a symbolic link after its target is renamed.

THIS LAB

Lab 3

First the concepts, with examples. Then the lab on the server. GRUB and shared libraries are optional extensions, so they come at the end.

DISCUSSION

You rename a report. What breaks?

A shortcut on the desktop, a second name for the same file, a note that says where the file is. Which of them still work?

CONCEPTS FIRST

Wildcards and links

Two ideas: the shell turns patterns into names, and one file can have more than one name.

A wildcard is a pattern, and the shell expands it

- A wildcard is a character that stands for other characters, such as * for any characters.
- The shell, not the command, expands the pattern into a list of matching names, then runs the command with that list.
- If nothing matches, Bash keeps the pattern as typed. The command then looks for a file with that exact name.
- Wildcards select file names. They are not the same as regular expressions.

BEFORE THE LAB

Why learn it here?

One pattern replaces a long list of names. The same pattern, used carelessly with rm, removes more than you meant.

The wildcard patterns

Pattern	Meaning	Example	Matches
`*`	Zero or more characters	`*.txt`	every .txt file
`?`	Exactly one character	`file?.txt`	file1.txt, fileA.txt
`[abc]`	One character from the set	`file[123].txt`	file1.txt, file2.txt, file3.txt
`[a-z]`	One character in the range	`[A-Z]*.txt`	names that start with a capital
`[!abc]`	One character not in the set	`file[!0-9].txt`	fileA.txt, not file1.txt
`{a,b}`	Brace expansion: generates words	`file.{txt,log}`	the words file.txt and file.log

Square brackets match one character that exists. Braces only write words, before any file is looked at.

Patterns at work

student@server: ~/oslab-work/lab3

```
$ ls *.txt
$ ls report0[1-3].txt
$ ls log_[!j]*.log
$ touch memo_{mon,tue,wed}.txt
$ cp *.csv archive/
$ ls *.tmp
$ rm *.tmp
```

The first three list files. touch with braces creates three files in one command.

cp copies every .csv file into archive/.

CAUTION

Preview before you delete

Run ls *.tmp first. If the list is what you expect, then run rm *.tmp.

See what the shell really passes

student@server: ~/oslab-work/lab3

```
$ mkdir -p practice
$ printf 'one\n' > practice/report1.txt
$ printf 'two\n' > practice/report2.txt
$ printf 'log\n' > practice/report.log

$ printf '<%s>\n' practice/*.txt
<practice/report1.txt>
<practice/report2.txt>
$ printf '<%s>\n' 'practice/*.txt'
<practice/*.txt>
```

printf prints each word it receives between angle brackets, one per line.

Without quotes, the shell sends two words: the two .txt files. With single quotes around the whole pattern, it sends one word with a literal *.

Use this trick to check a pattern before you give it to cp, mv or rm.

Brackets and braces are not the same

Square brackets look at files

```
student@server: ~/oslab-work/lab3

$ ls practice/report[12].txt
practice/report1.txt  practice/report2.txt
$ ls practice/report[13].txt
ls: cannot access 'practice/report3.txt': No such file
```

Only names that exist can match. Bash keeps a pattern with no match, so the error names the pattern.

Braces only write words

```
student@server: ~/oslab-work/lab3

$ echo practice/report{1,3}.txt
practice/report1.txt practice/report3.txt
```

The file report3.txt does not exist, and echo does not care. Brace expansion happens first, and no file is looked at.

A name is not the file

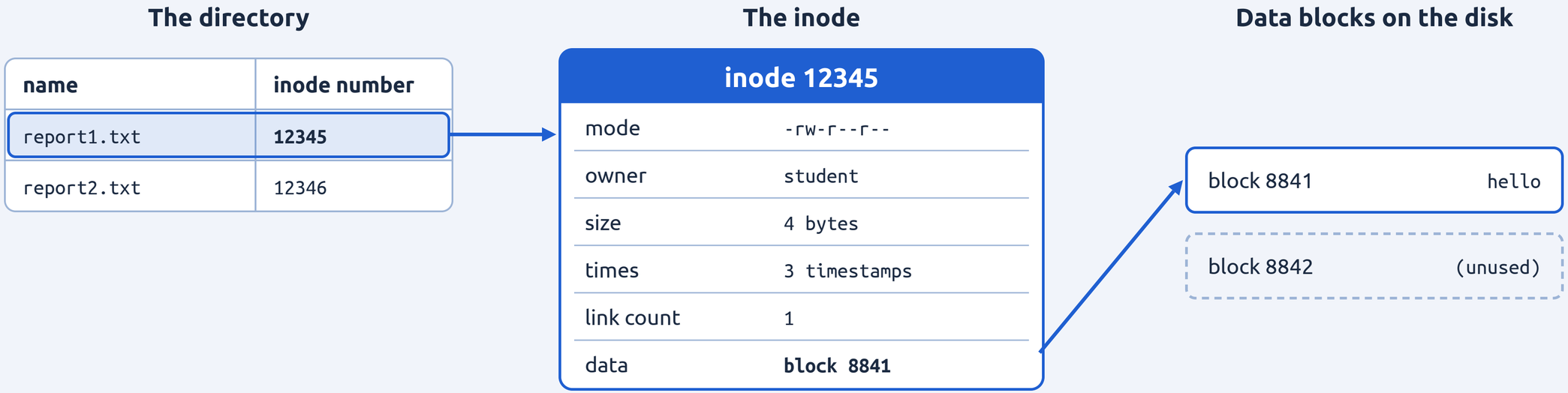
- The data of a file lives on the disk. An inode is the record that describes it: size, owner, permissions and where the data is.
- Every inode has a number. A file name is only an entry in a directory that points to an inode number.
- Two entries can point to the same inode. They are two names for one file: hard links.
- The data is freed only when the last name is gone.

inode number, permissions, link count, ...

```
$ ls -li
12345 -rw-r--r-- 2 student student 4 report1.txt
12345 -rw-r--r-- 2 student student 4 hard-demo.txt
99001 lrwxrwxrwx 1 student student 11 soft-demo.txt -> report1.txt
```

The 2 in the third column is the link count: two names point to inode 12345. The symbolic link has its own inode, 99001.

What an inode looks like



The directory maps a name to an inode number. The inode describes the file and says where its data is.

The name is not stored in the inode.

Read it from left to right: the name leads to an inode number, the inode leads to the data. ls -li shows that number.

Hard link and symbolic link

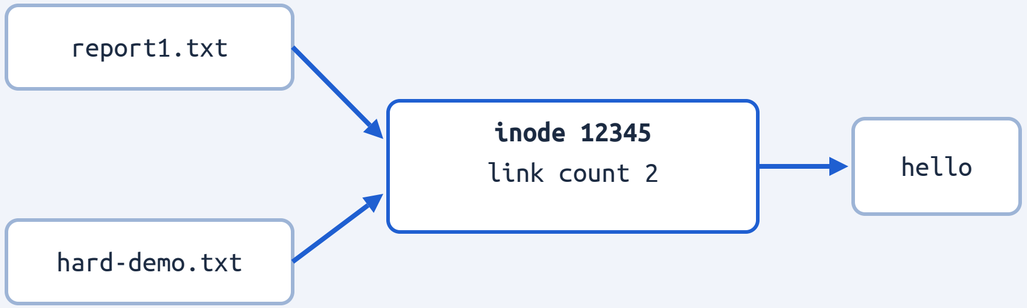
	Hard link	Symbolic link
Command	`ln target link`	`ln -s target link`
What it is	Another name for the same inode	A small file that stores a path
Inode	The same as the original	Its own inode
Original is removed	Data survives under the other name	The link breaks: it is dangling
Across filesystems	No	Yes
Folders	Not normally	Yes
Seen in ls -l	No arrow	An arrow to the stored target

Inspect with ls -li (inode numbers), stat (link count) and readlink (the stored path).

The two kinds of link, drawn

Hard link

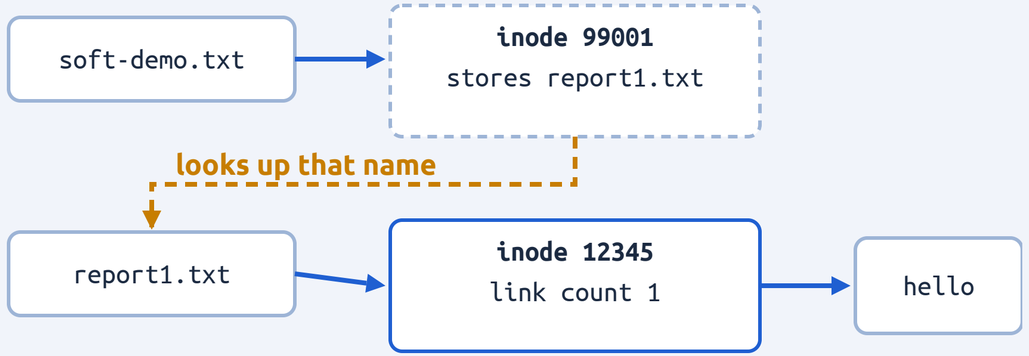
`ln report1.txt hard-demo.txt`



Two names for one inode.
Remove one name and the data stays.

Symbolic link

`ln -s report1.txt soft-demo.txt`



Its own inode, storing a path.
Rename the target and the path breaks.

A hard link is a second name for the inode. A symbolic link is a separate small file that holds a name to look up.

A relative target is read from the link's folder

student@server: ~/oslab-work/lab3

```
$ cd ~/oslab-work/lab3
$ ls -l links
lrwxrwxrwx 1 student student 11 soft.txt -> source.txt

$ cat links/soft.txt
version 1
$ cd links && cat soft.txt
version 1
```

The stored target source.txt has no / at the front, so it is relative.

It is resolved from the folder that holds the link (links/), not from the folder where you are standing.

That is why the same link works from two places. Write the target as if you were standing next to the link.

The key experiment

student@server: ~/oslab-work/lab3

```
$ echo 'config data' > original.txt
$ ln original.txt hardlink.txt
$ ln -s original.txt symlink.txt
$ ls -li
$ rm original.txt
$ cat hardlink.txt
config data
$ cat symlink.txt
cat: symlink.txt: No such file or directory
```

echo with > writes the text into a new file. rm removes only the name original.txt.

The data still has one name, hardlink.txt, so it survives.

symlink.txt still says original.txt, and no such name exists. The link is dangling.

In practice: ls -la /usr/bin/python* shows programs that are managed with symbolic links.

LAB 3

Wildcards and links

A file is renamed. The references that depend on it must still work, or you must say why they do not.

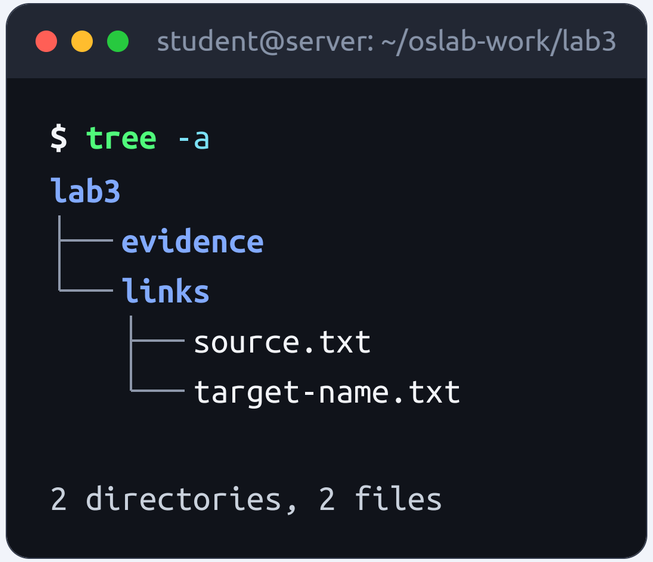
The scenario

SCENARIO

Alex at TechCorp

Alex maintains file references. A file is being renamed, and the references that depend on it must still work.

Wildcards and links are the core. Boot recovery and custom libraries are separate extensions.



Inside the 120 minutes

Minutes	What happens
0 to 10	Introduction, objectives and setup
10 to 25	Guided example: commands and observations
25 to 35	Prediction, saved with oslab predict
35 to 70	Rename, repair, and create a new file with the old name

Minutes	What happens
70 to 85	Check the results and fix incomplete work
85 to 100	Individual changed-case checkpoint, no AI or peers
100 to 110	Compare prediction with results, and explain
110 to 120	Submit the short report and evidence

Six helper commands, and why you use them

Command	What it does
`oslab prelab lab3`	Three short questions before class. A wrong answer costs nothing
`oslab predict lab3`	Saves your prediction once, before the experiment
`oslab hint lab3 1`	A hint. Use level 1, then 2, then 3. Hints cost no points
`oslab check lab3`	Checks your files and shows what is still missing
`oslab checkpoint lab3`	A short test near the end. It opens when the instructor says so
`oslab debrief lab3`	After the experiment: you give the right answers to your prediction questions

They are the same helper commands as in Labs 1 and 2. Before class, run oslab doctor and oslab prelab lab3. It takes about ten minutes.

How you work

Core, Plus, Challenge

Tasks 1 and 2 are the core, for everyone. Plus and Challenge are optional, for students who finish early. Full marks need only the core and the checkpoint.

AI tools

Allowed in the tasks, Plus and Challenge, so test what an AI tells you. Not allowed in the prediction and the checkpoint.

Your own account

Use only your own files and processes. No task in the core needs sudo, and privileged or boot work is never done on the shared server.

Your first prediction is not marked for being right. A reasoned attempt and an evidence-based correction earn credit.

Setup: start the lab

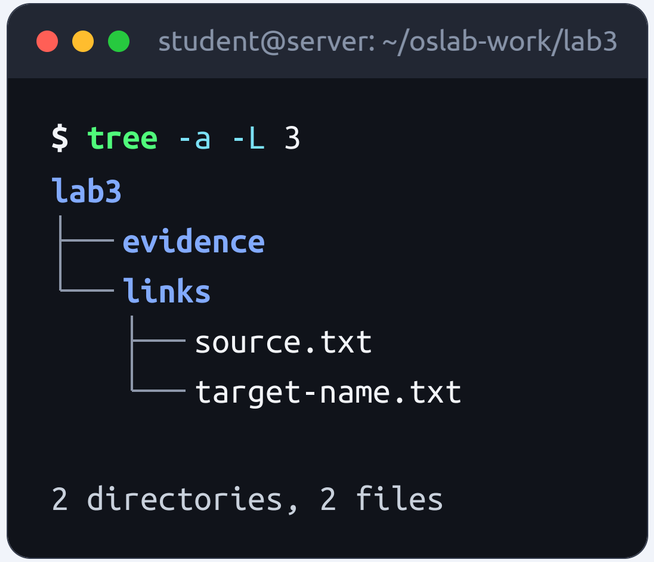
student@server: ~

```
$ export OSLAB_WORKSPACE="${OSLAB_WORKSPACE:-$HOME/oslab-work}"
$ oslab doctor
$ oslab start lab3
$ cd "$OSLAB_WORKSPACE/lab3"
$ pwd
$ mkdir -p evidence
$ find . -maxdepth 3 -type f
```

- oslab doctor must say class inbox: connected, so that your instructor receives your answers.
- If it says practice mode, tell the instructor before the prediction. Your answers stay on your machine.
- oslab start lab3 makes the lab folder and keeps your work if you start again. You cd into it yourself.
- If you are resuming, look at the existing files before you run a command that creates or renames them.

If oslab is missing, follow the setup guide in the course repository. Do not reset the lab just to fix one link.

Compare with the starting tree



links/source.txt contains version 1. target-name.txt is a spare file that the core does not use.

The helper writes a hidden file, .oslab-managed.json. It marks the folder as yours. Leave it as it is.

evidence/ was made by mkdir -p. The workspace is for experiments. Your personal course repository holds the final submission.

GUIDED

Wildcards, then your first links

Task 1, with the instructor: ten to twenty-five minutes.

Step 1: three clearly different names

student@server: ~/oslab-work/lab3

```
$ mkdir -p practice
$ printf 'one\n' > practice/report1.txt
$ printf 'two\n' > practice/report2.txt
$ printf 'log\n' > practice/report.log
$ ls practice
report1.txt  report2.txt  report.log
```

Work in a separate practice folder, so the investigation files stay untouched.

printf with > writes one line into a new file. The names differ on purpose: two .txt files and one .log file.

Step 2: how the shell expands a pattern

student@server: ~/oslab-work/lab3

```
$ {
$   printf '<%s>\n' practice/*.txt
$   printf '<%s>\n' practice/report?.txt
$   printf '<%s>\n' 'practice/*.txt'
$ } | tee evidence/wildcards.txt
```

Run this before you make the demo links in step 3. The braces group the three commands, and tee shows their output and saves it.

* matches any run of characters, ? exactly one. Quotes around the whole pattern leave a literal *.

REPORT

Record in your report

Which two names does the first command print? Does the second print the same names? Why does the third print a * and not file names?

Step 3: your first two links

student@server: ~/oslab-work/lab3

```
$ ln practice/report1.txt practice/hard-demo.txt
$ ln -s report1.txt practice/soft-demo.txt
$ ls -li practice/report1.txt practice/hard-demo.txt practice/soft-demo.txt
$ readlink practice/soft-demo.txt
```

A hard link names the same inode, on one filesystem. A symbolic link stores a path. ls -l shows the stored target.

readlink prints the path stored in a symbolic link.

TRY

Check together

Compare the inode numbers of report1.txt and hard-demo.txt, then the symbolic link's. Why is report1.txt, not practice/report1.txt, the right target here?

PREDICT

Guess first, then try

Before you rename the investigation source. Ten minutes.

The prediction

PREDICTION

You predict this sequence

In links/, source.txt holds version 1. You make hard.txt with ln and soft.txt with ln -s. You rename source.txt to renamed.txt. Later you create a new source.txt that holds version 2.

HOW

The rules

Alone, and without AI, before Task 2. Your first prediction is saved and sent to the instructor. A wrong guess costs nothing.

student@server: ~/oslab-work/lab3

```
$ oslab predict lab3
$ oslab status lab3
```

The helper asks whether each link works after the rename, which version the hard link reads after the new file, and one sentence of reasoning.

Do not create or upload a separate prediction file. oslab status lab3 confirms that it was saved.

TASK 2

Hard links and symbolic links

Thirty-five minutes, from minute 35 to 70. Work alone.

Step 1: two references to the source

student@server: ~/oslab-work/lab3/links

```
$ cd "$OSLAB_WORKSPACE/lab3/links"
$ cat source.txt
$ ln source.txt hard.txt
$ ln -s source.txt soft.txt
$ {
$   printf '\n=== BEFORE RENAME ===\n'
$   ls -li source.txt hard.txt soft.txt
$   printf 'hard.txt: '; cat hard.txt
$   printf 'soft.txt: '; cat soft.txt
$   printf 'stored target: '; readlink soft.txt
$ } | tee ../evidence/links.txt
```

tee starts evidence/links.txt. Later commands append to it with tee -a. The labelled line marks the state.

Check: all three names read version 1, and source.txt and hard.txt have equal inode numbers.

CAREFUL

If ln says File exists

Look at the existing entry first. Do not overwrite work blindly.

Step 2: rename the original, then look

mv renames the directory entry. -- means no more options follow. 2>&1 puts the error message into the saved evidence.

student@server: ~/oslab-work/lab3/links

```
$ mv -- source.txt renamed.txt
$ {
$   printf '\n=== AFTER RENAME, BEFORE REPAIR ===\n'
$   ls -li renamed.txt hard.txt soft.txt
$   printf 'hard.txt: '; cat hard.txt
$   printf 'stored target: '; readlink soft.txt
$   printf 'soft.txt: '; cat soft.txt
$ } 2>&1 | tee -a ../evidence/links.txt
```

THINK

The error is expected

cat soft.txt fails on purpose. The link still exists, but the path it stores names no file: a dangling symbolic link.

REPORT

Record in your report

What does readlink soft.txt print now, and which file name is missing? Answer for this state, before any repair.

Step 3: repair only the symbolic link

What to do

1. Look at the old link first: readlink soft.txt.
2. Remove only that owned entry, then create the link again with a relative target.
3. Do not delete or recreate hard.txt. Do not rename renamed.txt back.
4. Run cat soft.txt. It must print version 1.

YOUR TURN

You write the target

Nobody gives you the path. Replace TARGET with the path from the link's folder, links/, to the renamed file. In the guided part the link practice/soft-demo.txt stored report1.txt.

student@server: ~/oslab-work/lab3

```
$ rm -- soft.txt
$ ln -s TARGET soft.txt
$ cat soft.txt
```

Step 4: save the repaired state

student@server: ~/oslab-work/lab3/links

```
$ {
$   printf '\n=== AFTER REPAIR ===\n'
$   ls -li renamed.txt hard.txt soft.txt
$   printf 'stored target: '; readlink soft.txt
$   printf 'hard.txt: '; cat hard.txt
$   printf 'soft.txt: '; cat soft.txt
$ } | tee -a ../evidence/links.txt
```

The braces group the commands, so one tee records all of their output. -a appends to the same file.

TRY

Check

renamed.txt and hard.txt still share an inode. The repaired symbolic link reads version 1 through its stored path.

Step 5: a different file with the old name

student@server: ~/oslab-work/lab3/links

```
$ printf 'version 2\n' > source.txt
$ {
$   printf '\n=== NEW FILE USING THE OLD NAME ===\n'
$   ls -li source.txt renamed.txt hard.txt soft.txt
$   printf 'source.txt: '; cat source.txt
$   printf 'hard.txt: '; cat hard.txt
$   printf 'soft.txt: '; cat soft.txt
$ } | tee ../evidence/replacement.txt
```

Only renamed.txt, hard.txt and the repaired soft.txt refer to the original report now. This is a new file under the old name.

REPORT

Record in your report

The inode numbers of the new source.txt and of hard.txt, and the version each prints. Why does a new file under the old name not change what hard.txt refers to?

Complete when, and hints

DONE

Complete when

The hard link survives the rename. The repaired symbolic link resolves correctly. Your changed-name test tells a file name from an inode.

Hints: oslab hint lab3 1, 2, 3

1. A hard link is another directory entry.
2. Compare ls -li with readlink.
3. Resolve the symbolic target from links/, not from the folder where you are standing.

Check your work

Fifteen minutes, from minute 70 to 85.

Check your work

student@server: ~/oslab-work/lab3

```
$ cd "$OSLAB_WORKSPACE/lab3"
$ oslab check lab3
```

Before the checkpoint it checks four milestones: the wildcard output, the hard-link identity, the repaired symbolic target, and the new source's own inode and content.

Read each TRY line, fix that item, and run the check again. The result goes to the instructor, but it does not award your whole grade.

What your evidence should show

State in your evidence	Expected result
Before rename	Both links read version 1. Source and hard link share an inode
After rename, before repair	The hard link reads version 1. The symbolic link is dangling
After repair	The symbolic link resolves to renamed.txt and reads version 1
After creating a new source	The new source reads version 2. Both links still read version 1

The checker reads your current files. The labelled sections in evidence/links.txt show the earlier failure and repair, and your teacher reviews those. Keep links inside the lab folder.

Links and the helper

Internal links are fine

The helper accepts symbolic links that stay inside the managed folder.

Escaping links are refused

It refuses links that point outside it. Do not link to /etc, other homes or unrelated folders.

oslab clean lab3

Removes only the managed folder. Internal links go as entries, and external links are refused.

File exists means: look at the destination first. A dangling link can exist even when its target does not. Hard links cannot normally cross filesystems.

Plus: one more wildcard (optional)

student@server: ~/oslab-work/lab3

```
$ cd "$OSLAB_WORKSPACE/lab3"
$ printf '<%s>\n' practice/report[12].txt
$ printf '<%s>\n' practice/report[!1].*
$ printf 'Plus: my answer is ...\n' > evidence/plus.txt
```

[12] matches one character, 1 or 2. [!1] means any one character except 1.

Write down which names you expect from each command before you run it. Then add one sentence about any difference.

PLUS

Hint

The second pattern needs one character, then a dot, then anything.

Only when the core is finished and oslab check lab3 shows no TRY line. Not needed for full marks.

Challenge: a chain of links (optional)

student@server: ~/oslab-work/lab3

```
$ mkdir -p extra
$ cd extra
$ printf 'chain end\n' > end.txt
$ ln -s end.txt middle.txt
$ ln -s middle.txt start.txt
$ readlink start.txt
$ readlink -f start.txt
$ cat start.txt
$ cd ..
```

readlink prints one step. readlink -f follows the whole chain to the end.

CHALLENGE

Predict first

What happens to cat start.txt if you rename end.txt? Test it inside extra/ only, then add your answer to evidence/plus.txt.

Your core files stay as the checker expects them: Plus and Challenge use extra/ and evidence/plus.txt.

INDIVIDUAL CHECKPOINT

Show what you can do alone

Fifteen minutes, from minute 85 to 100. No AI, no neighbours.

Why there is a checkpoint

It checks you, not the tools

Everything before it may use AI and help. The checkpoint shows what you understood without them.

A short test in the terminal

oslab checkpoint lab3 opens when the instructor says so. Your earlier work does not need to be finished.

Your own case

The helper gives you your own names. The first answer in class is kept. In local practice it opens at once, and the teacher does not receive it.

What happens, step by step

1. The instructor opens it
Close your AI tools

→

2. oslab checkpoint lab3
Two short answers, one explanation

→

3. Follow the printed task
A small file task with your names

→

4. oslab check lab3
A fifth milestone checks it

While you work

Use your terminal and what you learned. No AI tools, no neighbours.

Your Task 2 files

Do not change them. The checkpoint describes a separate folder.

When you finish

oslab check lab3 shows whether the fifth milestone passes. You can fix and check again.

A practice example

PRACTICE

Resolve the path

A folder lab3/checkpoint-maple/ contains note.txt. A link shortcut.txt in that same folder stores ../note.txt, but lab3/note.txt does not exist. Does the link work? What relative target should it store instead? Which folder is the starting point for resolving the stored path?

What to give

1. Whether the link works.
2. The target it should store.
3. The folder the stored path starts from.

The public example is practice. The helper supplies the names for your own case.

Look back, then hand in

Ten minutes of correction, ten to clean up and submit.

Debrief: correct your prediction

```
student@server: ~/oslab-work/lab3

$ oslab debrief lab3
```

1. It asks the prediction questions again, after the experiment.
2. Your original prediction stays unchanged. Your corrected answers go to the instructor.
3. In README.md end with confirmed or corrected, and one sentence linking your prediction to an observed result.

REPORT

Your report answers

(a) Task 1: which names each pattern prints, and why the quoted one prints a *. (b) After the rename: what readlink prints and which name is missing. (c) The inode numbers and versions of source.txt and hard.txt, and why the new file does not change hard.txt.

Use your actual file names, inode numbers and version text. No general essay is needed.

Two places your work goes

What	Where it goes	How
Prediction, check results, checkpoint answers and debrief	The instructor's class inbox	Sent by the oslab commands. Confirm with oslab status lab3
The short report and three text evidence files	lab3/ in your personal course Git repository	Copy the files, then commit and push

Your oslab records are already collected. Do not copy private record folders, and do not submit screenshots of every command.

Submit 1: your repository and the copy

student@server: ~

```
$ SUBMISSION_REPO="$HOME/os-gic-YOUR_ID/os-lab-YOUR_ID"
$ mkdir -p "$SUBMISSION_REPO/lab3/evidence"
$ cd "$OSLAB_WORKSPACE/lab3"
$ cp -- evidence/wildcards.txt evidence/links.txt "$SUBMISSION_REPO/lab3/evidence/"
$ cp -- evidence/replacement.txt "$SUBMISSION_REPO/lab3/evidence/"
$ oslab status lab3
```

Replace YOUR_ID with your own ID. The orange text is the part you change. Use the absolute path of your existing course repository, never a neighbour's.

Example: with the ID 0123, the first line becomes SUBMISSION_REPO="$HOME/os-gic-0123/os-lab-0123", and echo "$SUBMISSION_REPO" prints /home/student/os-gic-0123/os-lab-0123.

Copy the three evidence files. oslab status lab3 shows what the class inbox already has. Keep the practice workspace until grading is done.

Submit 2: the final tree and Git

student@server: ~/os-gic-YOUR_ID/os-lab-YOUR_ID

```
$ tree
lab3
├── evidence
│   ├── links.txt
│   ├── replacement.txt
│   └── wildcards.txt
└── README.md

1 directory, 4 files
```

1. Write README.md from the lab's template, in your own words.
2. The prediction must have been written before you ran anything. Copying it into the README only records it.
3. git status --short, then add only your lab files, then commit and push.
4. No passwords, personal shell files, generated binaries or disk images.

How it is marked: 10 points

Evidence for the lab objectives	Points
Wildcard result, hard-link identity and repaired symbolic target: 1 point each	3
Labelled rename, failure and repair evidence, and the recreated-name comparison: 1 point each	2
Original prediction saved, then a debrief with an evidence-based correction: 1 point each	2
Individual checkpoint: path answers and a working repaired link: 1 point each	2
Complete short report and the three evidence files submitted	1

A helper PASS alone does not prove your understanding: the teacher reads your short explanations and the labelled failure and repair record. A wrong first prediction is not penalized. Equivalent valid target paths earn credit. If you use AI, add one sentence naming a suggestion and how you checked it.

Common mistakes

Mistake	What you see	Fix
Running ln again on an existing name	File exists	Look at the destination first
Target written from where you stand	A dangling link	Write it as if you stood next to the link
Replacing hard.txt to fix the link	The experiment loses its point	Repair only soft.txt
A bare pattern with rm	More files removed than expected	Run ls with the same pattern first
Brackets used like braces	A name that does not exist	[ ] match files, { } write words
Predicting after the rename	No prediction saved in time	Run oslab predict lab3 before Task 2

GRUB and shared libraries

Optional extensions. They need a disposable virtual machine or a compiler, and they are not part of the two-hour core.

From power-on to login

Stage	What it does
BIOS or UEFI	Self-test, then finds the boot device
GRUB	Shows a menu and loads the kernel
Kernel	Starts the hardware, mounts the root filesystem, runs the first process (PID 1)
init or systemd	Starts the system services in order
Login	User space is ready

GRUB files and key settings

File	Purpose	Edit?
`/boot/grub/grub.cfg`	The generated menu	Never
`/etc/default/grub`	Timeout, kernel parameters	Yes
`/etc/grub.d/`	Scripts, custom entries	Yes
`/boot/vmlinuz-*`	The compressed kernel	No

Setting	Meaning	Example
`GRUB_DEFAULT`	Entry to boot	0
`GRUB_TIMEOUT`	Seconds at the menu	10
`GRUB_CMDLINE_LINUX_DEFAULT`	Kernel parameters	quiet splash
`GRUB_BACKGROUND`	A menu background image	/boot/grub/bg.png

After you edit /etc/default/grub, run sudo update-grub. Custom entries go in /etc/grub.d/40_custom. Do this only in a disposable virtual machine, never on the shared server.

GRUB recovery

Mode	How you enter	Use
Recovery mode	GRUB menu, Advanced options, recovery	A root shell, fsck, limited services
GRUB command line	Press c at the menu	Boot by hand when the menu is broken
Live USB	Boot from an external image	Full rescue when GRUB is gone

grub>

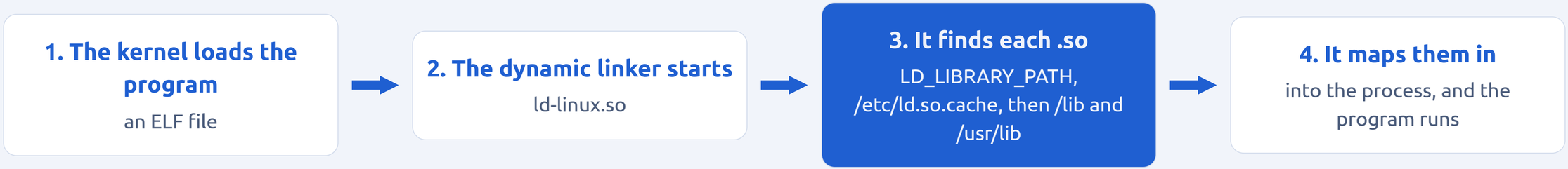
```
$ set root=(hd0,msdos1)
$ linux /boot/vmlinuz-VERSION root=/dev/sda1
$ initrd /boot/initrd.img-VERSION
$ boot
```

Take a snapshot of the virtual machine before you break anything on purpose.

Static and dynamic linking

	Static	Dynamic
When linked	At compile time	When the program starts
Library code	Copied into the executable	Loaded from .so files
Executable size	Larger	Smaller
Memory	Each program has its own copy	Programs share one copy
Updates	Recompile every program	Replace the .so, and all programs benefit
File extension	.a (archive)	.so (shared object)

How a program finds its libraries



Looking at shared libraries, and building one

student@server: ~/oslab-work/lab3

```
$ ldd /bin/ls
$ ldconfig -p | grep libc
$ gcc -fPIC -c mylib.c -o mylib.o
$ gcc -shared -o libmylib.so mylib.o
$ gcc main.c -L. -lmylib -o myprogram
$ LD_LIBRARY_PATH=. ./myprogram
```

ldd lists the libraries a program needs. ldconfig -p searches the library cache.

-fPIC makes position-independent code, so the library can load at any address. -shared makes the .so file.

For a user-local library, set LD_LIBRARY_PATH instead of installing into system folders.

More ways to look at a library

student@server: ~/oslab-work/lab3

```
$ file /lib/x86_64-linux-gnu/libc.so.6
$ readelf -h /bin/ls
$ grep '[.]so' /proc/$$/maps
$ sudo ldconfig
```

file tells you the kind of file: an ELF shared object.

readelf -h prints the ELF header of a program. /proc/$$/maps lists what the current shell has loaded.

After you install a new .so in a system folder, sudo ldconfig refreshes the cache. Not on the shared server.

Key takeaways

01

Wildcards

The shell expands a pattern into names before the command runs. Quote a pattern to keep it literal.

02

Name and inode

A file name is an entry that points to an inode. Hard links are more names for one inode.

03

Symbolic links

A symbolic link stores a path. A relative target is read from the link's folder.

04

Evidence

Rename, look, repair, replace. Each step has a record you can check.

Next session

COMING UP

Lab 4: I/O redirection, pipelines and processes

Send output to files, connect commands with pipes, and see and stop processes.

SOURCE

Reading

Silberschatz, Galvin and Gagne, *Operating System Concepts*, 10th edition, chapters 13 and 14, File-System Interface and File-System Implementation.